House Price Prediction Using Machine Learning: A Study on the Impact of Spatial and Environmental Features

Yuyang Zhang
School of Computer Science
University of Nottingham Ningbo China
Ningbo, China
Student ID: 20514470
scyyz26@nottingham.edu.cn

***Abstract*—Predicting house prices is one of the main tasks in real estate research. However, house price data are characterized by non-linearity, spatial dependency, and heavy tails; moreover, no model has been found to perform better than others in all cases. One can use stacking ensembles to combine complementary algorithms. Still, the approach to predicting house prices is inconsistent in the literature, and most studies do not reveal the contribution of each base learner when building an ensemble. In this paper, an end-to-end pipeline is developed on the King County housing data (4,600 observations, 18 variables) to answer the following questions: does stacking yield better results compared to the best-performing individual model, and which base learners have the most influence on performance improvements? To begin with, six base learners are evaluated against the same preprocessing procedure: Ridge, Random Forest, Extra Trees, GBR, XGBoost, and Multi-Layer Perceptron (MLP). Then, three stacking schemes are built on the same base learners using out-of-fold predictions and either Ridge or LassoCV as the meta-model. The best stacking model yields R^2 = 0.7429, which is only marginally better than the best base model, XGBoost (R^2 = 0.7375). Leave-one-out ablation shows that Random Forest (+0.0030) and XGBoost (+0.0011) are mostly responsible for performance improvements; other base learners make no substantial contribution, and Ridge even harms ensemble performance. Therefore, increasing the number of base learners might not be the right way to improve ensemble performance since it is mostly determined by several strong learners and spatial features coded as the target variable mean.**

***Index Terms*—house price prediction, stacking ensemble, ablation study, target encoding, gradient boosting**

I. Introduction

House prices are a crucial concern for buyers, sellers, lenders, and policymakers [1], and they are notoriously difficult to model. Each property constitutes an almost unique combination of structures, location, and condition, where the relationship between each factor and price is nonlinear and imbalanced across the market [2]. Methods based on additive linear relationships, like hedonic regression and other parametric models, struggle to account for the interactions and give inconsistent prices when transaction numbers are sparse. [1].

Machine learning techniques relax these assumptions and are now ubiquitous methods for property valuation. Nevertheless, none of these methods performs consistently across all datasets and markets. Ridge, linear models in general, are easy to interpret but incapable of modeling non-linearities [3]. On the other hand, tree-based ensembles perform well in modeling interactions but suffer from over-fitting and interpretability issues [4], [5] . Deep networks are highly flexible but architecture choices and tuning affect results cite 10984370,JIT2774. This problem motivates stacking, where a meta-model combines the outputs of several heterogeneous learners so as to keep the best of each one. Yet, stacking is not extensively used in real estate price prediction due to inconsistent advantages reported so far [6]. Most of the related literature uses single learners and simple combinations of learners, such as bagging and boosting algorithms [5]. The few studies that use stacking show some gains but treat the ensemble as a black-box approach with little discussion of what each of the base learners contributes to the prediction task [7] . Without this understanding, it is uncertain if the improvements generalize or a simpler ensemble would work just as well.

This study tried to fill the gap using the King County housing dataset (4,600 transactions, 18 features, May 2014 to May 2015) [8]. In particular, it combines structural attributes (number of bedrooms, bathrooms, and square feet of living space), environmental characteristics (waterfront and view), and high cardinality location features (city and state/zip code), which allows various learners to take advantage of each piece of information differently. Contributions are:

- A controlled comparison of six baseline regressors and three stacking strategies with identical preprocessing.
- A leave-one-out experiment analyzing how important each base learner is to the stacked model through its contribution to the final prediction, aided by meta-learner coefficients.
- Identify that stacking produces the most value in Random Forest and XGBoost, and that target means of location features are dominant in feature importance.

II. Literature Review

Data-driven approaches to estimating the value of houses can be classified into three broad groups: parametric baselines, single-learner machine learning, and ensemble models.

For parametric baseline models, Hedonic regression models assume a linear relationship between the price and characteristics of a property [1]. They are straightforward and cost-effective to apply, but due to their linear assumption, they are susceptible to non-linearity and spatial heterogeneity in the data [3], [9].

As machine learning gained traction in this area, a range of single-learner models has been applied to housing valuation. Tree-based learners, decision trees [4], random forests [10], and gradient boosting [11], [12] — are the most widely used, because they do not require feature scaling and capture feature interactions without explicit transformation of the target. Neural networks have outperformed linear baselines in some comparisons [9] but are prone to overfitting and offer limited interpretability [3]. Support vector regression with a non-linear kernel has also been applied, but is sensitive to the choice of C and the ϵ-insensitive loss [13]. The recurring conclusion across these studies is that any of these learners can be made to work well on a given dataset with careful tuning, but no single family is consistently best.

This is the gap that ensemble methods are designed to close. Bagging and boosting reduce the variance of individual learners by aggregating many of them [10], [11]. Stacked generalisation goes further by training a meta-learner on the predictions of heterogeneous base models [14], and is in principle better placed than a single ensemble family to exploit complementary errors. In the housing-valuation literature, however, stacking remains relatively rare. The studies that do use it tend to focus on overall accuracy without controlled comparisons against individually tuned baselines [6], [15], and rarely decompose the ensemble to show which base learners actually contribute. Bjørgve et al. [7] note this explicitly, arguing that the practical case for stacking in housing valuation is still weak because gains are often modest and the underlying mechanism is opaque.

III. Methodology

The pipeline has six stages: exploratory analysis, cleaning, feature engineering, feature selection, baseline modelling, and stacking. Each stage is described below, with leakage-aware design choices flagged where they apply.

A. Exploratory Analysis

The dataset is loaded from `data.csv` and inspected for missing values, dtype consistency, and target skew. No imputation is needed. The price distribution is right-skewed with a long upper tail, so outlier handling is required before training error-sensitive metrics such as RMSE. Continuous predictors are visualised against price (regression plots), categorical predictors via boxplots, and pairwise relationships via a correlation heatmap.

B. Outlier Removal

Price observations beyond the 99th percentile are dropped (46 observations, 1.0% of the data). Such a decision removes a few extremely valuable and luxury houses that would dominate model fitting and RMSE objective.

C. Feature Engineering

The `date` column is used to derive temporal features, including *house_age*, *is_renovated*, and *years_since_renovation*, which capture the age and renovation history of the property. The feature *basement_ratio* represents the relative size of the basement, while *has_basement* is a binary indicator of basement presence. A small constant is added to avoid division by zero when computing the ratio. The feature *bath_bed_ratio* contains information on the number of bathrooms and bedrooms, providing additional structural information. The *month* variable is converted into seasonal categories and subsequently encoded using one-hot encoding.

D. Encoding and Train–Test Split

After the application of feature engineering techniques, raw identifiers (*street*, *city*, and *statezip*) are dropped. The data are split into training (80%, 3,643 instances) and test (20%, 911 instances) sets using a random split with a fixed seed for reproducibility. To avoid the dimensional growth and at the same time preserve location information, the categorical variables *city* and *statezip* are reintroduced using target encoding. Each category is replaced by the mean house price computed from the training data. For categories in the test set that are not observed during training, the global mean of the training set is used. All encoding statistics are computed exclusively on the training data to prevent data leakage.

E. Feature Selection and Scaling

The feature selection step combines automatic and manual techniques. Features are selected automatically based on the criteria of correlation with price: $|r| > 0.05$. In addition, a manual set of domain-relevant features is saved regardless of their univariate correlation. Features with high mutual correlation, $|r| > 0.85$, among other retained features are pruned. Two exceptions from this rule are *sqft_living*, *bathrooms*, and the target-encoded location columns. As a result of this operation, only 14 features are left (dropped is *basement_ratio*). Remaining features are standardized based on training data only.

F. Baseline Models

Six baseline models are grouped into three categories: regularized linear, bagging, and boosting ensembles. Linear ridge regression acts as the representative of the first group. Bagging ensembles include Random Forest and Extra Trees. GBR and XGBoost stand for boosting ensemble techniques. Finally, the neural network regressor (MLP) provides non-linear representation capabilities. All models share the same scaled feature set and are evaluated with R^2, RMSE, and MAE.

G. Stacking Ensemble

Out-of-fold (OOF) predictions are generated for each base learner using 5-fold cross-validation. For each fold, predictions on the held-out subset are used to construct meta-features, while test predictions are averaged across folds. Three stacking configurations are then trained on these meta-features. Configuration (**V1**) uses only the OOF predictions, utilizes a Ridge meta-learner. In the second configuration (**V2**), the meta-features are concatenated with the original, scaled features before feeding into the Ridge meta-learner. Lastly, the third configuration (**V3**) substitutes LassoCV for the meta-learner, thus introducing automatic regularization and feature selection.

H. Evaluation

All models are assessed using the held-out 20% test split. To reach the second research goal, a leave-one-out ablation on the top-performing stacking approach is performed, whereby the impact of each base learner on the test R^2 is measured by removing one base learner at a time. Additionally, meta-learner coefficients for V1 and V3 are provided for two reasons. First, they show how the predictions from different base learners are combined and indicate any redundancy in the models through correlation of the coefficients. Second, for configurations what including Ridge meta-learner, V1 outperforms V2(See details in Results). Furthermore, Random Forest feature importances and three diagnostic plots (predicted-versus-actual, residuals against predictions, residual histogram) are used to inspect bias and heteroscedasticity.

IV. Experimental Results

The results will be presented in the same order as the pipeline above: observations from EDA, followed by an evaluation of all models, an ablation study, meta-learner coefficients, and diagnostics.

A. EDA and Feature Engineering Insights

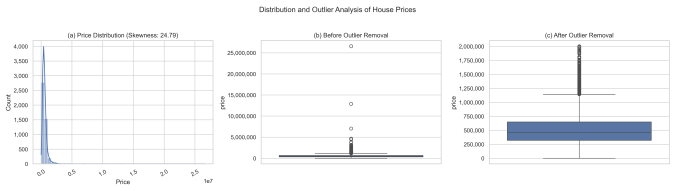

Fig. 1: Price distribution and outlier analysis. (a) Right-skewed original distribution. (b) Boxplot before outlier removal. (c) After dropping the top 1% (46 rows), the bulk of the distribution is far more compact.

The distribution of house prices indicates the need for outlier handling. As shown in Fig. 1a, the price distribution is extremely right-skewed, with the majority of observations concentrated at lower price levels and a small number of extreme values extending to the right tail. This pattern is further illustrated by the box plot in Fig. 1b, where numerous high-value outliers are visible. After removing the top 1% of observations, the distribution becomes more compact, as shown in Fig. 1c.

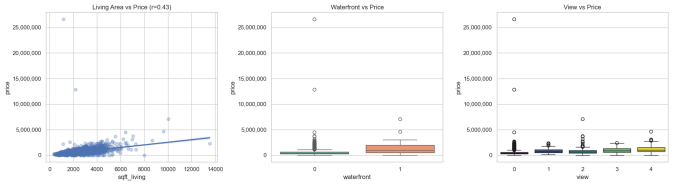

Fig. 2: Key predictors against price. (a) Living area is positively correlated with price. (b) Waterfront properties command a clear premium. (c) Higher view ratings track higher prices monotonically.

Fig.2 depicts the three main factors that guided our feature engineering efforts. Panel (a) shows a plot between *sqft_living* and price, showing that there is an increasing trend, as well as increasing variation for higher values of this feature, implying a non-linear relationship. Panel (b) indicates that properties that have waterfronts form a group with higher prices. Panel (c) shows that higher *view* ratings are associated with progressively higher prices, suggesting a monotonic relationship between view quality and property value.

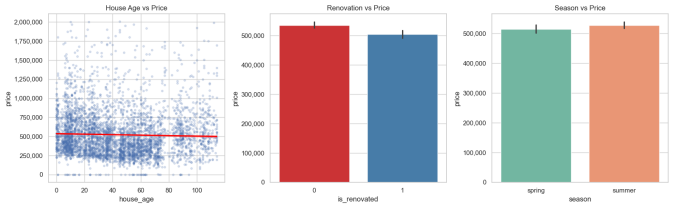

Fig. 3: Engineered features against price. House age, renovation status, and season individually carry weak signal but contribute non-additively to ensemble models.

These constructed features, as shown in Fig. 3, have very little predictive power by themselves. The correlation shown in Fig. 3a is a weak negative one between the age of the house and the price. Fig. 3b shows that there is only a slight difference between the prices of houses that have been renovated and those that have not been. Fig. 3c shows that there is not much variation depending on the season.

City effects (Fig. 4) are sizable and heavy-tailed, as just a few cities are responsible for the higher end. With numerous cities included in the dataset, one-hot encoding would be redundant since it increases the dimensionality of the feature space without adding any value.

The ranking of correlation shown in Figure 5 gives the inputs that have contributed the most to the signal. The top three predictors include zip_te (r=0.668), sqft_living (r=0.640), and city_te (r=0.532). The secondary set of predictors comprises bathrooms (r=0.489), bedrooms (r=0.327), and view (r=0.321). While the engineered features rank relatively low, they are kept according to the rules outlined above. The importance of both the location variables encoded by target

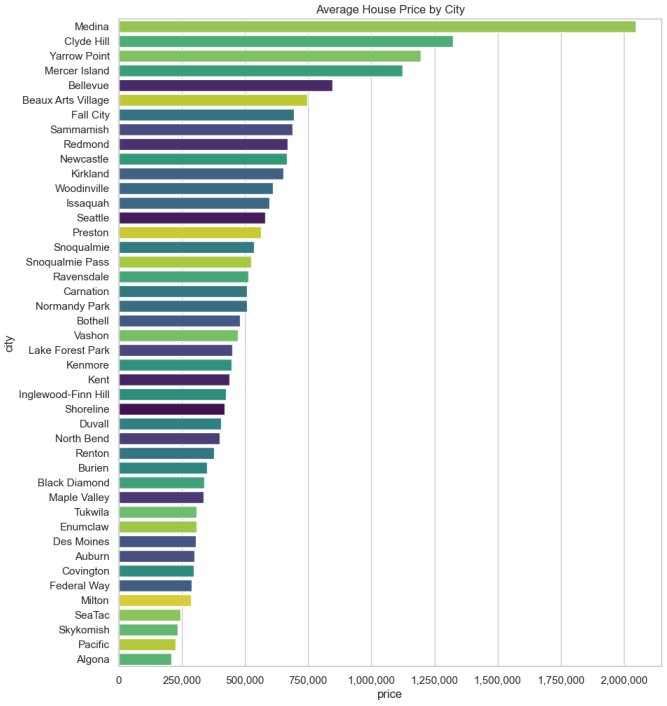

Fig. 4: Average price by city. The distribution is long-tailed: a few cities account for the high end of the market, justifying target encoding rather than one-hot.

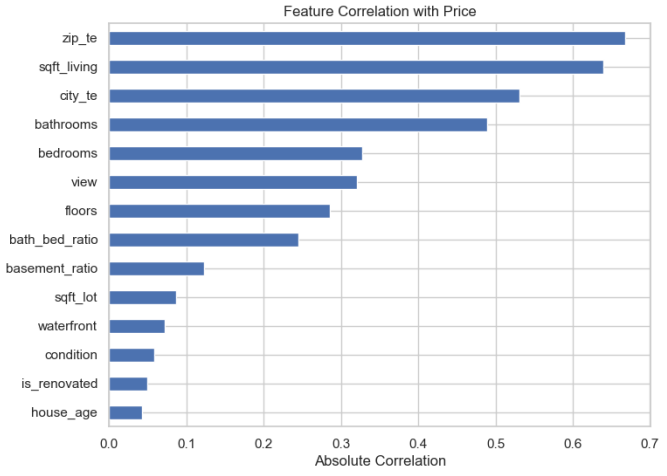

Fig. 5: Absolute correlation of each retained feature with price. *zip_te*, *sqft_living*, and *city_te* are the three strongest individual predictors.

encoding stands out as a common theme for the rest of the analysis.

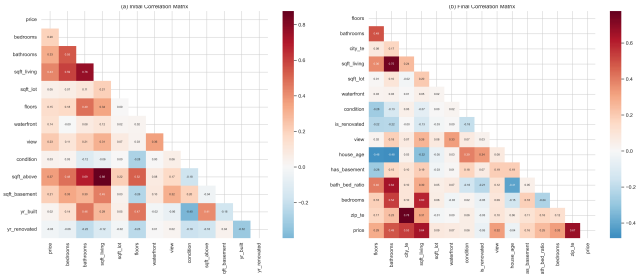

Fig. 6: Correlation matrix before and after feature selection. (a) Strong multicollinearity among size-related features(left). (b) After dropping *basement_ratio*, redundant pairs are removed and relationships with the target are preserved(right).

The before-and-after comparison (Fig. 6) illustrates the effect of the multicollinearity reduction phase. Only the predictor basement_ratio is eliminated, leaving a 14-predictor model in which all the meaningful correlations are retained. This process reduces redundancy among highly correlated features while preserving the primary predictive relationships with the target variable. Therefore, linear models are robust in further analysis.

B. Model Comparison

TABLE I: Test-set performance of all nine models, sorted by R^2.

Model	R^2	RMSE	MAE
Stacking V1	0.7429	147405	88005
Stacking V2	0.7428	147437	88232
Stacking V3	0.7422	147608	88009
XGBoost	0.7375	148950	88548
GBR	0.7298	151094	89896
Random Forest	0.7276	151724	88525
MLP	0.7046	158004	99704
Extra Trees	0.7026	158540	94566
Ridge	0.7002	159160	100678

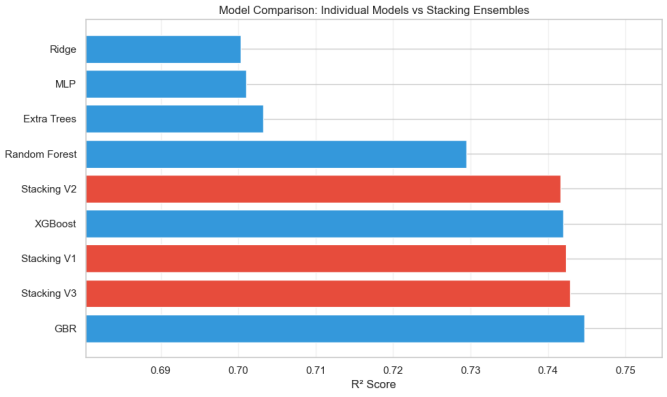

Fig. 7: Test R^2 for all nine models. Stacking variants narrowly beat the best individual model.

The comparative performance between the models is shown in Table I and Fig. 7. From the individual models, XGBoost performs the best (R^2 = 0.7375) compared to GBR and Random Forest. The models using boosting and bagging algorithms rank high among the individual models, while the linear and neural baseline models are ranked relatively low. The result aligns with the findings from the exploratory data analysis (EDA), showing that the most important features have non-linear relationships with each other. All stacking configurations outperform XGBoost with a margin of superiority in terms of R^2 and RMSE. Among the three configurations, V1 shows the highest R^2 score of 0.7429. The difference in R^2 and RMSE scores between the highest performing model (V1) and XGBoost is about 0.005 and 1,500, respectively. Although the margins are small, the improvements show consistency in all three performance measures and all three stacking configurations, suggesting that the differences are not accidental. Comparing V1 to the other two stacking configurations, we find that Configuration V1 performs better with an advantage in terms of R^2 of less than 0.001.

C. Ablation and Meta-Learner Coefficients

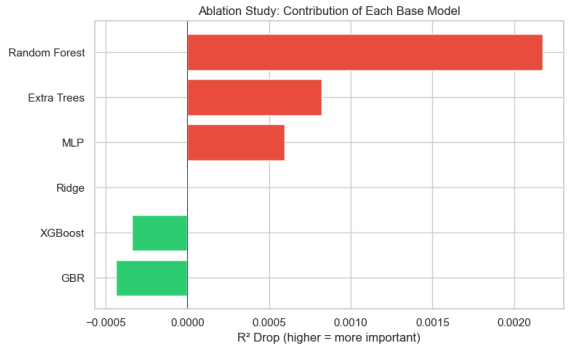

Fig. 8: Leave-one-out ablation on Stacking V1. A positive ΔR^2 means removing the model hurts performance.

TABLE II: Leave-one-out ablation on Stacking V1.

Removed model	Resulting R^2	ΔR^2
Random Forest	0.7399	+0.0030
XGBoost	0.7418	+0.0011
Extra Trees	0.7424	+0.0005
GBR	0.7427	+0.0002
MLP	0.7428	+0.0000
Ridge	0.7433	−0.0004
None (full)	0.7429	0

As can be seen from the ablation results in Fig. 8 and summarized in Table II, the hierarchy is obvious: the main player here is Random Forest, whose elimination causes a decrease in R-squared value of $\Delta R^2 = 0.0030$, about three times greater than the one caused by elimination of XGBoost ($\Delta R^2 = 0.0011$). GBR, MLP, and Extra Trees' contributions do not exceed 0.0005, and removing Ridge leads to the marginal improvement of 0.0004. This hierarchy, as expected, is supported by the V1 coefficients presented in Table III.

TABLE III: Meta-learner coefficients on the OOF predictions. V1 uses Ridge; V3 uses LassoCV (the Extra Trees coefficient was set to exactly zero by Lasso).

Base learner	V1 (Ridge)	V3 (Lasso)
Random Forest	+0.5630	+0.4503
XGBoost	+0.2610	+0.2612
Ridge	+0.2415	+0.2277
GBR	+0.0802	+0.0900
MLP	+0.0172	+0.0093
Extra Trees	−0.1306	0 (dropped)

The largest coefficients correspond to the most important base models: Random Forest has a positive coefficient of +0.563, and XGBoost has a positive coefficient of +0.261. Ridge also gets a significant coefficient: +0.242. GBR and MLP get smaller values: +0.080 and +0.017, respectively, whereas Extra Trees gets a negative −0.131 value. This does not mean that an isolated, Extra Trees model would perform worse. It uses the same algorithm as Random Forest, but the split criteria are different. As a consequence, the correlation between Extra Trees' and Random Forest's predictions is high, so that the Ridge learner uses Extra Trees in order to remove the overlap and leave only disagreement, which is the goal of the stacking technique.

Another piece of evidence supporting the idea of redundancy of Extra Trees comes from the meta-Lasso in the V3 setting. When allowed to drop predictors, it gives zero weight to Extra Trees, while the other five base models obtain nearly equal coefficients to those obtained previously in V1. Since the new model gets almost an equal R-squared value ($R^2 = 0.7422$) compared to V1's result ($\Delta R^2 = 0.0008$), it shows that Extra Trees is redundant given that Random Forest is included in the ensemble. Thus, the conclusion we can draw based on the above results and analysis is that stacking mainly leverages Random Forest and XGBoost, whereas other learners have marginal effects, and Extra Trees turns out to be fully redundant in the presence of the Random Forest model.

D. Diagnostics and Without Preprocessing Training

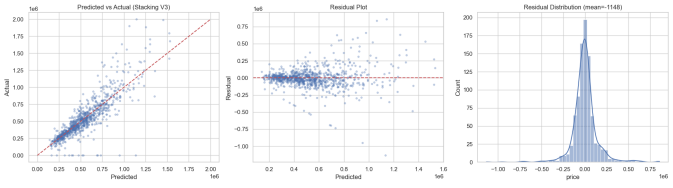

Fig. 9: Diagnostics for Stacking V3: predicted vs actual, residuals vs predicted, and residual distribution.

The results shown in Figure 9 confirm what is presented in the table. As can be seen from Fig. 9a, the predictions are tightly clustered around the line of identity. Fig. 9b shows that the residuals are centered around zero but their variance increases with increasing predicted values, which implies some degree of heteroscedasticity since the model has lower precision when predicting the prices of more costly

properties, a result that is not unexpected since there were few observations left after filtering out those beyond the 99th percentile threshold.

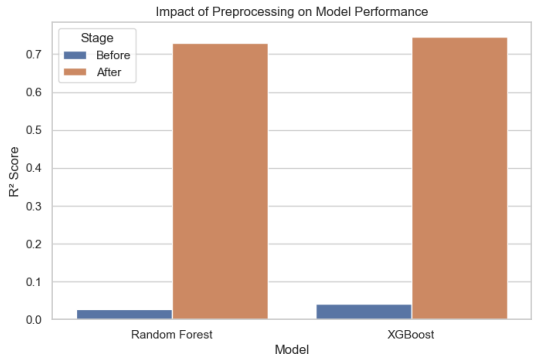

Fig. 10: R^2 for Random Forest and XGBoost before and after applying the preprocessing pipeline.

Figure 10 shows the effect of pre-processing. Without pre-processing, both Random Forest and XGBoost produce an R^2 on the test set of about 0.03. After the pre-processing step is applied to the data, the Random Forest produces a score of 0.723 and the XGBoost model 0.733. The difference in the results is considerably greater than the best-to-worst result difference shown in Table I.

V. Discussion

VI. Discussion

The outcomes show that ensemble models, and especially those based on trees, better fit the structure of housing data compared to linear or neural models. This can be seen from the ranking provided in Table I: XGBoost, Gradient Boosting Regressor (GBR), and Random Forest are among the top three, while Ridge regression and the MLP are at the bottom of the list. This outcome is coherent with the EDA results: the relationship between *sqft_living* and price is non-linear; waterfront and view variables are responsible for price steps; and target-encoded location features represent categorical information about price quite well. Linear models do not capture this non-linearity due to the additive assumption, which is not satisfied for housing data, while the MLP requires a larger sample and/or more expressive features to outperform the boosting baselines [11], [12].

Furthermore, stacking provides a further improvement, even though it is minor. The three stacking variants exceed XGBoost by about 0.005 in R^2, an increase consistently observed across all three stacking configurations. This phenomenon is clarified by the ablation analysis. Among the six learners, Random Forest and XGBoost provide the most valuable input to the ensemble; Extra Trees mainly decorrelates Random Forest, while the contribution of the remaining three learners is minimal. As a result, the meta-learner does not integrate six different perspectives on the problem but rather reweights two tree-based perspectives. This outcome is consistent with the findings of Bjørgve et al. [7], according to which the gains obtained through stacking in housing valuation are moderate when the strongest base learner is not the right direction to optimize, as base -learner diversity is what really counts.

A second observation concerns how the contribution of each base learner varies depending on whether it works in isolation or stacked with other learners. For example, XGBoost outperforms Random Forest by 0.010 in R^2 as a standalone model, but when included in the stacking ensemble, Random Forest contributes nearly three times as much as XGBoost (Table II). This indicates that the contribution made by a base learner to the ensemble is not determined by its standalone performance, but by the complementary information it carries relative to the other learners. The OOF predictions of XGBoost apparently overlap with those of GBR and Extra Trees, which use related algorithmic mechanisms. On the contrary, Random Forest contributes a pattern difficult to replicate by the meta-learner using other learners' predictions. Such information cannot be captured by performance metrics; that is why leave-one-out ablation helps identify it.

The last two comments apply to all model comparisons performed during this research project. First, the analysis of the preprocessing variant shows that the choice of data preparation strategy has a greater impact on model test performance than model selection itself: from $R^2 \approx 0.03$ to $R^2 > 0.72$ upon removing outliers and engineering features. This outcome implies that, contrary to the conventional practice, preprocessing should not be considered an automatically performed preliminary step but rather regarded as a modelling decision. Second, the dominance of the target-encoded location features in the correlation ranking, *zip_te* (r=0.668, r=0.668, r=0.668) and *city_te* (r=0.532,r=0.532, r=0.532) relating to *sqft_living* (r=0.640, r=0.640) r=0.640), confirms the importance of spatial information in housing valuation. It can be explained by the well-known problem of ignoring spatial heterogeneity in hedonic models [9].

VII. Conclusion and Future Work

This paper presents a stacking ensemble for house price prediction through a controlled comparison. In this study, we analyze stacking for house price predictions through a controlled experiment on the King County Housing dataset. Our optimal stacking setup reached $R^2 = 0.7429$, barely beating XGBoost ($R^2 = 0.7375$). In addition, the ablation analysis shows that the contribution to the result from our stacking setup is virtually entirely made by Random Forest and XGBoost. With the Lasso meta-learner, it also seems clear that this result would be nearly identical if the last four algorithms were left out. This means that there are just two learners worth including in the stack and adding other learners does not contribute much. This suggests, practically, that optimizing stack performance should involve choosing good base learners, not increasing the stack size.

Two additional results follow from this analysis. The first is that preprocessing has a larger impact than model selection on this dataset. Tree-based learners move from $R^2 \approx 0.03$ on raw data to $R^2 > 0.72$ once outliers are removed and engineered

features are added. This shows that the signal connecting features to price exists in the dataset, but only becomes accessible to the model after preprocessing. The second result is that the target-encoded location features (*zip_te*, *city_te*) rank among the strongest predictors, on a par with *sqft_living*. Therefore, geography accounts for a substantial share of the price variance, and target encoding offers a low-cost way to capture this signal without the dimensionality cost of one-hot encoding.

This study has three main limitations. First, the model performs poorly on luxury houses due to insufficient representation by 99th- percentile filtering and a lack of suitable features. Second, we use purely structural tabular features to make predictions. Including some other signals like economic factors, schools, geographic coordinates, etc., could have provided further gains. Third, adding more learners did not lead to significant performance boosts, as the gains seem to come mostly from a few good learners. These limitations point to three directions for future research. The first one is to train the model with custom loss functions designed to handle the right part of the distribution and/or weight high-cost cases. This could mitigate the heteroscedastic issue cite hjort2022house. The second option is to experiment with different types of geographic encoding, like learned embeddings or coordinate features, instead of using target-encoding. Finally, instead of doing ablation studies after meta-learner fitting, we need a principled method for selecting base learners based on diversity.

REFERENCES

[1] N. Kok, P. Monkkonen, and J. M. Quigley, "Land use regulations and the value of land and housing: An intra-metropolitan analysis," *Journal of Urban Economics*, vol. 81, pp. 136–148, 2014. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0094119014000278

[2] S. Rosen, "Hedonic prices and implicit markets: Product differentiation in pure competition," *Journal of Political Economy*, vol. 82, no. 1, pp. 34–55, 1974. [Online]. Available: http://www.jstor.org/stable/1830899

[3] V. Limsombunchao, "House price prediction: Hedonic price model vs. artificial neural network," *Agricultural Economics Research Unit Working Paper*, 2004.

[4] G.-Z. Fan, S. E. Ong, and H. C. Koh, "Determinants of house price: A decision tree approach," *Urban Studies*, vol. 43, no. 12, pp. 2301–2315, 2006.

[5] J. R. Rico-Juan and P. Taltavull de La Paz, "Machine learning with explainability or spatial hedonics tools? an analysis of the asking prices in the housing market in alicante, spain," *Expert Systems with Applications*, vol. 171, p. 114590, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417421000312

[6] W. Kim and J. Hong, "Stacked ensemble model for the automatic valuation of residential properties in south korea: A case study on jeju island," *Land*, vol. 13, no. 9, 2024. [Online]. Available: https://www.mdpi.com/2073-445X/13/9/1436

[7] E. Bjørgve, A. Oust, A. J. Pollestad, C. Sandnes, and O. J. Sønstebø, "Comparing housing valuation techniques and stacked generalization: Exploiting explainable ai," *The Journal of Real Estate Finance and Economics*, vol. 72, pp. 640 – 665, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:281736517

[8] Kaggle, "House sales in king county, usa," https://www.kaggle.com/datasets/harlfoxem/housesalesprediction, 2015, accessed: 2026-05-02.

[9] H. Selim, "Determinants of house prices in turkey: Hedonic regression versus artificial neural network," *Expert Systems with Applications*, vol. 36, no. 2, Part 2, pp. 2843–2852, 2009. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417408000596

[10] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, p. 5–32, Oct. 2001. [Online]. Available: https://doi.org/10.1023/A:1010933404324

[11] A. Hjort, J. Pensar, I. Scheel, and D. E. Sommervoll, "House price prediction with gradient boosted trees under different loss functions," *Journal of Property Research*, vol. 39, no. 4, pp. 338–364, 2022. [Online]. Available: https://doi.org/10.1080/09599916.2022.2070525

[12] J. H. Friedman, "Greedy function approximation: A gradient boosting machine." *The Annals of Statistics*, vol. 29, no. 5, pp. 1189 – 1232, 2001. [Online]. Available: https://doi.org/10.1214/aos/1013203451

[13] V. Cherkassky and Y. Ma, "Practical selection of svm parameters and noise estimation for svm regression," *Neural Networks*, vol. 17, no. 1, pp. 113–126, 2004. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0893608003001692

[14] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0893608005800231

[15] S. Ganapathy and T. Li, "An explainable ai framework with ml model stacking for house price prediction," in *2024 IEEE MIT Undergraduate Research Technology Conference (URTC)*, 2024, pp. 1–5.